

SOCIAL NETWORK PLATFORM SUPPORTING ENGLISH LEARNING AND MULTIMEDIA COMMUNICATION (Nov 2025)

Nguyen Thi Hoang Khanh, Truong Quoc Bao, Nguyen Thanh Thuan

Faculty of Information Technology Industrial University of Ho Chi Minh City
nguyenthihoangkhanh@iuh.edu.vn, tqbao44@gmail.com, nguyenthuan020010@gmail.com

Abstract— In the context of the growing prevalence of online learning, integrating social elements into foreign language learning environments has become an inevitable trend. However, most existing English learning platforms still lack meaningful learner interaction, fail to leverage multimedia effectively, and provide limited personalization in learning pathways. This paper introduces Social-Learning, an English-learning social network platform developed using a Client-Server architecture. The platform seamlessly combines learning features (listening, writing, and speaking practice, personalized vocabulary building, and learning path recommendations) with social networking functionalities (posting, commenting, making friends) and multimedia communication tools (messaging, voice calls, video calls). The system is built with Next.js, Node.js, Express.js, React Native, Supabase, and MongoDB, aiming to deliver a dynamic, interactive, and accessible learning environment. The primary goal of the platform is to enhance learner motivation, strengthen community engagement, and expand the potential applications of artificial intelligence in personalizing English learning content.

Index Terms—Social learning network, English learning, Cross-platform application, Supabase, Vocabulary personalization, Learning path recommendation.

I. INTRODUCTION

Online language-learning platforms have experienced a significant surge in popularity amid the digital transformation, with over 500 million users worldwide. Nevertheless, the majority of such applications revolve around individualized and automated learning, which leaves a gap: the absence of socializing and the actual communicative setting. This drawback tends to lessen the motivation of the learners and does not exploit the potential of the multimedia tools to their full potential [1]. Educational studies, specifically Vygotsky (1978), have indicated that community-based learning, as well as social interaction, are imperative in order to promote the effectiveness of learning and long-term learning habits [2]. In order to overcome these hurdles, the current paper presents a social-network-based application called Social-Learning, which facilitates the process of learning English and multimedia communication. The system is developed in a manner that helps combine a structured learning environment and a loose social network where the learners can not only practice the skills, but they can also connect, share, and interact within a global community. The platform is built on a cross-platform Client-Server architecture (Web and Mobile) and is

developed on modern technologies of Next.js and React Native on the Client side and Node.js/Express.js on the Backend with a hybrid database system built on Supabase (PostgreSQL) and MongoDB.

The virtuosity of the Social-Learning is the concept of the profound combination of the social (posting, making friends), real-time, multimedia (messaging, voice calls, video calls) communication features, and a complete complex of learning tools. The system under consideration is oriented to the customization of the learning process with the help of writing, listening, and speaking practice, the mechanism of managing vocabulary automatically, and the suggested learning directions. The platform will empower users to monitor their progress, get feedback, and engage in leaderboards, which will help to enhance motivation and stick to the habit of regular learning. Also, administrators are offered monitoring and analytics tools in the system. The paper is divided into five parts, and section one is a background of the situation and solution. Section 2 talks about the theoretical background and associated technologies. Section 3 outlines the system architecture and functional subsystems of the system called Social-Learning. Section 4 records the testing procedure and experiment assessment. Lastly, Section 5 provides conclusions and future development directions.

II. THEORETICAL BACKGROUND

A. TypeScript

TypeScript is an open-source software that was created by Microsoft, and it is regarded as a better version of JavaScript. It introduces functionality that JavaScript does not have, such as optional static typing and object-oriented classes. Applications on the client side (e.g., Angular 2) and the server side (with Node.js) can be developed in TypeScript. It uses the ECMAScript 2015 (ES6) features of the language, and is constantly undergoing updates to adopt the current techniques of future versions of ECMAScript [3].

B. Next.js

Next.js is an open-source platform created by Vercel and published in 2016. It is an open-source tool that serves React-based web apps that are generated with the use of a static

site and can be rendered on the server side to improve user experience, web performance, and search engine visibility [4].

C. React Native

React Native is a JavaScript framework that is applied to create native mobile applications on iOS and Android. It is grounded on the Facebook React library, but unlike that, designed to work on mobile platforms rather than on web browsers. This enables web developers to develop applications using JavaScript that appear native in the look and feel. Its best feature is that it can even share a large portion of the source code between both platforms, and thus it becomes extremely easy to develop Android and iOS simultaneously [5].

D. Gemini API

The Google Gemini API is a highly powerful tool that most developers in the modern world can use in software, applications, and small businesses. The Gemini API supports text and image input and, therefore, gives the user insightful and context-aware responses with the ability to reason intelligently [6].

E. Supabase (PostgreSQL)

Supabase is an open-source Backend as a Service (BaaS) platform, which includes a set of tools that help to create applications quickly without the need to manage the servers manually. It is based on PostgreSQL [7] as a central database and also provides characteristics like real-time database, user authentication, file storage, and serverless functions. Supabase is considered to be an open-source Firebase substitute, in general [8].

F. Client-Server

Client Server Architecture. The client-server architecture is a distributed network architecture where the client devices make service requests to a centralized server, whereby the server processes the request and responds accordingly. The server stores resources, and the data processing is done by the server, the clients access and use these services, which can be either a computer or a mobile device. Interaction between the two is based on networking protocols like TCP/IP in order to promote effective communication [9].

G. Accuracy

The system measures pronunciation accuracy [10] on a scale of a variant of the Word Error Rate (WER) measure. The Speech-to-Text API responds by giving the user a text string (the actual string) when a phrase is pronounced. The comparison between this string and the original one (the expected string) is made to find out the number of Substitutions (S), Deletions (D), and Insertions (I). The accuracy is computed as follows:

$$Accuracy = 1 - \frac{S + D + I}{N} \cdot 100\%. \quad (1)$$

In which N represents the number of words in the expected string. Within the Personal Speaking Practice, 100% Accuracy is required to complete the system successfully.

H. Mastery Score

The Mastery Score is a dynamic score that is updated after every vocabulary review session. The mastery level M of a vocabulary item is 0 to 100, and it can be determined as follows:

- When the user answers correctly:

$$M_{new} = M_{old} + P_{correct} \quad (2)$$

- When the user answers incorrectly:

$$M_{new} = \max(0, M_{old} - P_{wrong}) \quad (3)$$

In this case, M_{new} is the new mastery value, M_{old} is the old mastery value, $P_{correct}$ and P_{wrong} are the reward and penalty weights (e.g., +5 on each correct and 3 on each incorrect answer). This process helps to make sure that a user will have to practice a certain vocabulary item several times before he or she can understand it [11].

III. RELATED WORK AND SYSTEM COMPARISON

This study proposes the Social-Learning Platform (SLP) as a solution to overcome the limitations of existing language-learning applications through three core components: real social interaction, an AI-driven adaptive learning loop, and a flexible system architecture. This section provides a detailed comparison between SLP and widely used platforms such as Duolingo, ELSA Speak, and Quizlet.

A. Feature and Technology Comparison

Table I summarizes the differences in learning approaches between SLP and current solutions, emphasizing the role of Gemini API integration in enabling deep personalization.

B. Key Differentiators

1) *Social and Multimedia Integration:* While platforms such as Duolingo emphasize individualized and automated learning, SLP positions itself as a comprehensive social learning network.

Community Environment: SLP provides friend connections, posting, commenting, and resource-sharing functionalities, forming a realistic and engaging learning community.

Real-Time Communication (RTC): SLP supports real-time multimedia communication, including messaging, voice calls, and video calls. This design aligns with educational research indicating that social interaction is essential for maintaining learning habits, enhancing motivation, and developing soft skills while reducing geographical barriers.

2) *Deep AI-driven Adaptive Learning Loop:* SLP incorporates the Gemini API to build a dynamic adaptive loop that surpasses traditional static learning exercises.

Error-triggering mechanism: The system tracks user mistakes across all activities (listening, speaking, writing). When a word reaches five logged errors, it becomes a new learning resource.

Contextual expansion: AI expands each error word with synonyms, definitions, and contextual examples, ensuring meaningful vocabulary acquisition rather than isolated memorization.

3) *Accessibility and System Architecture*: Accessibility: SLP aims to provide an affordable and accessible platform, contrasting with the premium and paywall-restricted models of applications such as ELSA and Duolingo.

Scalability: The platform is built on a modern Client–Server architecture (Next.js, React Native) with a hybrid database layer (PostgreSQL & MongoDB) using Supabase. As an open-source backend, Supabase provides self-hosting capabilities and greater flexibility compared with proprietary systems such as Firebase.

TABLE I: Feature Comparison Between Social-Learning Platform (SLP) and Competitors

Feature	SLP	Competitors
Learning Path Recommendation	Adaptive: AI generates a personalized learning path (time frame, skills) based on user goals and learning history, and updates continuously according to real performance.	Static/Input-based: Duolingo/ELSA provide paths determined by a placement test; limited ability to adapt weekly based on diverse performance data.
Writing Practice	Deep Analysis: Uses Gemini API to evaluate grammar, sentence structure, coherence, and provides holistic scoring with detailed improvement suggestions.	Basic: Traditional apps focus on quizzes or fill-in-the-blank; tools like Grammarly correct errors but do not integrate with personal vocabulary tracking.
Vocabulary Management	Automated Tracking: System automatically adds words to a personal vocabulary list once the user makes five errors. AI expands each item with synonyms, definitions, and contextual examples.	Manual: Quizlet/Memrise require users to manually add words or rely on existing flashcard sets, lacking error-triggered automatic vocabulary generation.
Speaking Practice	Fluency & Context: Focus on contextual fluency with 80% pronunciation threshold to ensure natural interaction in real-time AI mode. Support answer suggestions, grammar and sentence structure evaluation, and overview comments	Accuracy-only: ELSA emphasizes phoneme-level accuracy but lacks natural context-based conversational interaction.

IV. SOCIAL-LEARNING PLATFORM FOR ENGLISH LANGUAGE ACQUISITION AND MULTIMEDIA COMMUNICATION

A. System Architecture Overview

The Social-Learning platform is architected as a modern, distributed system leveraging a client-server model to ensure scalability, maintainability, and cross-platform compatibility. The system integrates real-time communication capabilities with AI-driven learning modules, supported by a hybrid database infrastructure. Figure 1 illustrates the high-level architecture, which comprises three primary layers: the Frontend Client, the Backend Core, and the External Services & Data Layer.

1) *Frontend Client Layer (Cross-Platform Interface)*: To maximize accessibility, the client-side is developed using a unified codebase philosophy where applicable, targeting both mobile and web users:

- **Mobile Application**: Built with React Native, the mobile app provides a native experience for iOS and Android

users. It handles device-specific features such as microphone access for speaking practice and camera integration for video calls.

- **Web Application**: Developed using Next.js, the web platform offers a responsive and SEO-friendly interface. It utilizes server-side rendering (SSR) to optimize initial load times and improve performance for learners accessing the platform via browsers.

Both client interfaces communicate with the backend via secure HTTP requests and WebSocket connections for real-time interactions.

2) *Backend Core Layer (Business Logic & Orchestration)*:

The backend serves as the central nervous system, built on the Node.js runtime environment and the Express.js framework. Hosted on a Digital Ocean VPS, this layer is responsible for:

- **API Management**: Handling RESTful API requests from clients, validating user inputs, and managing session authentication.
- **Orchestration**: Coordinating interactions between the database, file storage, and external AI services.
- **Real-time Processing**: Managing signaling for multimedia communication (video/voice calls) and instant messaging features via Socket.IO.

3) *Data & External Services Layer (Hybrid Infrastructure)*:

The system employs a polyglot persistence strategy and integrates specialized third-party services to handle diverse data types:

a) *Hybrid Database System*:

- **Supabase (PostgreSQL)**: Acts as the primary relational database for structured data, including user profiles, learning progress, social connections, and the “VocabErrors” repository. Supabase also manages authentication and authorization via Row-Level Security (RLS).
- **MongoDB**: Utilized as a NoSQL database for storing high-volume, unstructured data such as chat logs and activity feeds, ensuring efficient retrieval and horizontal scalability.

b) *AI & Cloud Services*:

- **Google Gemini API**: The core engine for the *Adaptive Learning Loop*, responsible for generating personalized learning paths, correcting writing exercises, and powering the AI conversation partner.
- **Google Cloud API**: Employed for specialized tasks such as Speech-to-Text conversion to evaluate pronunciation accuracy.
- **Cloudinary & Supabase Storage**: Managed media asset storage (audio, video, images), ensuring optimized delivery via Content Delivery Networks (CDNs).
- **ZEGOCLOUD**: Integrated SDK for high-quality, low-latency voice and video call functionalities.
- **Sepay**: Facilitates secure payment processing for premium features.

4) *Deployment & CI/CD*: The development lifecycle is streamlined using GitHub for version control. A Continuous Integration/Continuous Deployment (CI/CD) pipeline automates the deployment process: frontend updates are pushed to

distribution platforms, while backend changes are deployed to the Digital Ocean server, ensuring rapid iteration and stability.

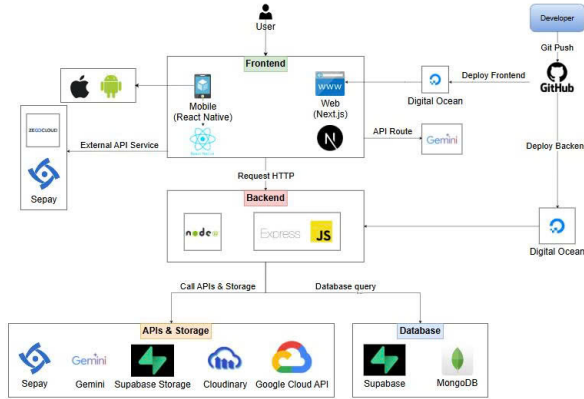


Fig. 1: System architecture diagram.

B. Authentication

To address the critical requirements of user data protection and system integrity, the Social-Learning platform implements a multi-layered security strategy rooted in the Supabase (PostgreSQL) ecosystem. The security mechanism is built upon three pillars: Secure Authentication via JWT, Database-level Authorization via Row-Level Security (RLS), and Strict API Key Management.

1) *Authentication and Session Management*: The system leverages Supabase Auth (GoTrue) to manage identities, ensuring that all sensitive credentials are handled securely.

a) *Password Protection*: All user passwords are automatically hashed and salted before storage, eliminating the risk of plain-text exposure.

b) *JWT Mechanism*: Session management relies on JSON Web Tokens (JWT). Upon successful login, the client receives a short-lived access token (default 1 hour) and a long-lived refresh token. This stateless approach allows for scalability while maintaining security via token revocation capabilities.

c) *Brute-force Mitigation*: To prevent unauthorized access attempts, the system enforces rate limiting on authentication endpoints. five consecutive failed login attempts trigger a mandatory 15-minute lockout period for the account.

d) *Multi-Factor Authentication (MFA)*: To meet non-functional security constraints, the architecture supports Time-based One-Time Password (TOTP) MFA, adding a critical layer of defense against compromised credentials.

2) *Authorization via Row-Level Security (RLS)*: Unlike traditional architectures that rely on application-level logic for data protection, Social-Learning enforces authorization directly at the database layer using PostgreSQL's Row-Level Security (RLS).

a) *Policy Enforcement*: RLS policies act as mandatory filters for every SQL operation (SELECT, INSERT, UPDATE, DELETE). For instance, a policy using the `auth.uid()` function

ensures that users can only access their own private vocabulary lists, even if the API endpoint is technically public.

b) *Granular Access Control*: By utilizing Custom Claims within the JWT (e.g., `user_role`), the system implements Role-Based Access Control. Policies are defined using `USING` clauses for existing rows and `WITH CHECK` clauses for new data modifications, ensuring that no unauthorized data manipulation can occur.

c) *Defense in Depth*: RLS is enabled on all tables by default. This ensures that even if the client-side code contains vulnerabilities, the data remains protected at the storage level.

3) Infrastructure and API Security:

a) *API Key Segregation*: The system strictly distinguishes between the `anon` key (public, client-side safe, restricted by RLS) and the `service_role` key (private, bypasses RLS). The `service_role` key is exclusively used in trusted server-side environments (Node.js backend) for administrative tasks and is never exposed to the frontend.

b) *Transport Security*: All data transmission between the Client (React Native/Next.js) and the Server is encrypted using TLS/SSL, preventing Man-in-the-Middle (MitM) attacks.

c) *Secret Management*: API keys and database secrets are managed via environment variables and are rotated periodically to minimize the impact of potential leaks.

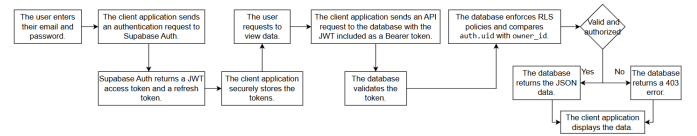


Fig. 2: Activity stream.

C. AI-Based Personalized Learning.

The use of traditional language-learning applications usually does not involve personalized feedback, which is an essential element of successful language learning. In order to overcome this shortcoming, the social-learning system incorporates the Google Gemini API, a state-of-the-art multimodal AI model, as the brainchild of its interactive learning capabilities. We do not simply stick to predefined exercises but instead make use of AI to create dynamic learning experiences that can analyze, respond, and interact in real time. The implementation of this integration is in the form of the Node.js backend that acts as a middleman and handles the request of the client (Next.js and React Native) and interacts with the Gemini API. The workflow is implemented on five major characteristics: (1) AI-based writing practice and automatic scoring, (2) listening generated by AI, (3) speaking practice based on AI and interactive conversations, (4) Individual vocabulary acquisition, and (5) Recommendations based on AI-driven learning paths.

1) *Writing Practice*: The Writing Practice feature of the social-learning system is created to assist users in improving their writing proficiency in English on different topics and at different levels. The students type their responses in written form on the interface provided by the system, where this information is sent to the server to be processed. The written material is sent to the Gemini API on the backend, where

it is analyzed in terms of grammar, sentence structure, usage of vocabulary, and general coherence. The AI then provides feedback and details, including:

- Overall Score: Calculated based on grammatical accuracy, vocabulary richness, and logical coherence.
- Improvement Suggestions: Recommends better word choices or improved sentence structures.

This feedback is shown to the learners as a graphic interface, with the highlighted parts, and as a detailed explanation. Wrong words or phrases are also written into the personal error database (VocabErrors) to help in individual vocabulary learning in later modules. Also, the system enables learners to have an archive of their writing history, therefore, monitor their progress over the years, compare their average score, and access sample essays that suit the current level of their proficiency.

2) *Listening Practice* : The Listening Practice feature is developed based on the text-to-speech technology of Gemini API in combination with audio storage and delivery of Cloudinary. The following steps are in the workflow:

a) *Content Generation*: Gemini API creates brief conversations or listening texts, depending on the topic and the level of proficiency and skills (e.g., daily conversation, travel, academic contexts).

b) *Speech Synthesis*: The text thus created is converted to sounding natural audio, with the pacing and the intonation being modified to the level of the learner.

c) *Playback and Interaction*: Students are given audio and are asked to do tasks like fill-in-the-blank, multiple choice, or dictation.

d) *Scoring*: The system uses the responses of the learners against the original text, and gives accuracy scores and jots down the vocabulary/grammar errors.

e) *Data Storage*: The VocabErrors database stores results and error data to be analyzed later and provides a person with specific vocabulary advice. This can enhance the listening comprehension of the learners, besides offering them an enriched practice environment by automatically creating listening exercises, a choice of multiple voices, and control of speaking rates. The integration plays a key role in promoting natural exposure and standardization of pronunciation in the process of learning.

3) *Speaking practice*: The speaking practice feature is the only one that will assist the user in enhancing their pronunciation and speaking skills, as well as be used in diagnosing, gathering, and analyzing personal pronunciation mistakes. This data helps the platform to offer tailored learning paths. The system has two modes of practice, one of which serves a specific purpose:

a) *Personal Speaking Mode*: It is the mode that focuses on absolute pronunciation accuracy. There are 10 sample sentences, which are either static content or AI-generated in the system. To pass, the users have to pronounce every sentence correctly (100%). All words that are identified as mispronounced are automatically added to the personal database of the user (VocabErrors). This error data is used to: recommend individual exercises, track user progress, and streamline future practice sessions.

b) *AI-assisted Speaking Mode*: It is a mode that is interested in fluency and contextual conversational ability. Each user is shown a choice of role (A or B) and engaged in a 10-sentence dialogue that was produced by the Gemini API. The threshold passing mode is reduced in the current mode, which has natural communication; an example is that pronunciation accuracy of 80% must pass. Just like the personal mode, the words spoken out of pronunciation will be added to VocabErrors. This allows the learners to gain confidence in communication and, at the same time, trace mistakes in pronunciation as in the personal mode.

4) *Personalized Vocabulary* : The system does not force the users to recognize mistakes in their vocabulary, but rather interferes to assist them in learning the vocabulary. The process is as follows:

a) *Trigger*: An automated agent in the system is taken into action when a particular vocabulary word is logged into the database (VocabErrors) as an error and crosses a specified number (e.g., 5 instances).

b) *Vocabulary Set Generation*: The agent will automatically generate a custom vocabulary for the user.

c) *AI Expansion*: This list includes not only the original mispronounced or abused words. It is forwarded to the Gemini API (with a special prompt) to be enriched and expanded, including: synonyms and antonyms, IPA transcription, definitions, and example sentences. This makes the learner have a profound knowledge of each word and not learn it in isolation.

d) *Mastery Score System*: The word set of the personalized set has a mastery score attached to each word. The score rises as the user responds correctly or falls as the user responds incorrectly, hence a good representation of what the learner has acquired as knowledge over time. Vocabulary Practice: Besides the skill-based practice (listening, speaking, and writing), the system creates exercises depending on the individual vocabulary set, thus giving the learners an opportunity to study and solidify each word comprehensively.

5) *Path of Learning Recommendation*: To ensure technical feasibility and prevent the generation of non-existent content ("hallucination"), the recommendation engine operates on a Constrained Generative Model flow

a) *Data Collection and Constraint Injection*: The *Context Builder* module (Node.js) aggregates three distinct data sources to construct a strict input context:

- User Constraints (I_u): Includes *Skill Focus* (e.g., Speaking), *Target Study* (e.g., IELTS 6.5), *Field Focus* (e.g., Business/Academic), and critically, *Available Time* per day.
- Learning History (H): Derived from the user's *Current Level* and historical *Score Metrics* to determine the appropriate starting difficulty.
- System Syllabus Injection (S): A structured JSON representation of the available *ExerciseList* (containing valid Topics and Levels) is injected into the prompt. This constrains the AI to select only topics that actually exist in the system's database.

b) *Strict JSON Output Schema*: The Gemini API processes these inputs to generate a personalized roadmap. Unlike free-text generation, the system enforces a strict JSON schema

output. This ensures that the Frontend can deterministically parse the plan into actionable tasks.

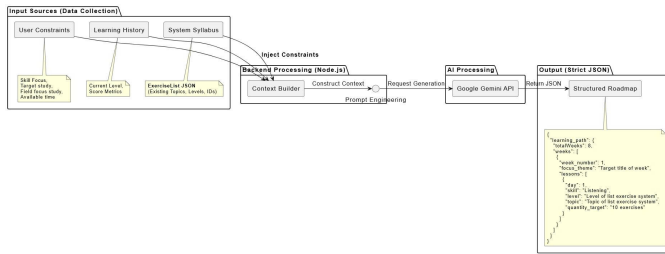


Fig. 3: Data flow of the AI-driven learning path recommendation. The diagram illustrates the "Constrained Generation" mechanism where the System Syllabus is injected into the prompt to prevent content hallucination.

c) *Feasibility and Adaptation*: By mapping the AI-generated *level* and *topic* fields directly to the system's exercise repository, the platform ensures 100% content availability. Furthermore, the roadmap is adaptive; as users complete the *quantity_target*, the system updates the *Learning History*, triggering a re-evaluation of the path for subsequent weeks.

V. IMPLEMENTATION AND RESULTS

A. Deployment



Fig. 4: System deployment diagram.

Description of deployment Diagram. Nginx is a reverse proxy server that forwards the client requests to the respective backend services.

- **Website Server**: Contains the user interface of the whole system.
- **Backend Server**: The backend server deals with all the business logic, and it communicates with the database to deal with user and system data.

VPS Server Configuration The server is based on a virtualized platform (VM) utilizing a QEMU virtual CPU based on an Intel CPU of x86-64 (64-bit) architecture. The system is programmed with 1 CPU/socket and 1 thread that offers one processing thread with a clock speed rate of 2.0 GHz. The virtual machine has complete support for VT-x virtualization technology. In respect of cache memory, the system has 64 KiB L1d/L1i and 8 MiB L2/L3 cache. It is worth noting that the operating system has allowed critical security mitigations (PTI and IBRS) to be used to defend against the usual hardware vulnerabilities, including Meltdown and Spectre.

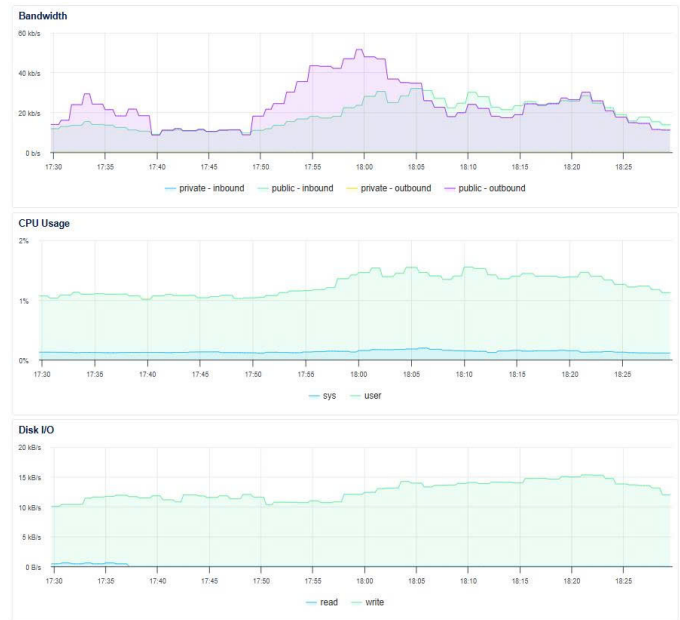


Fig. 5: VPS server configuration.

B. Implementation of User Interface.

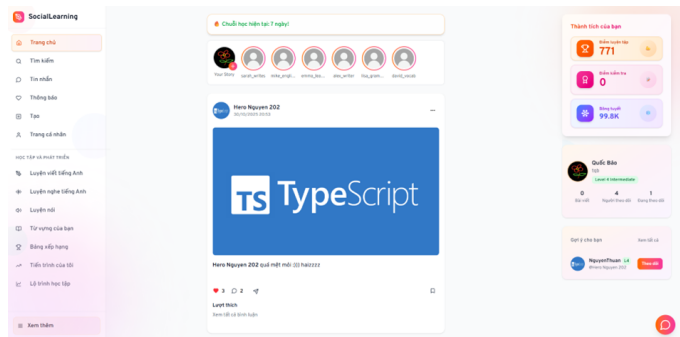


Fig. 6: Website user interface.

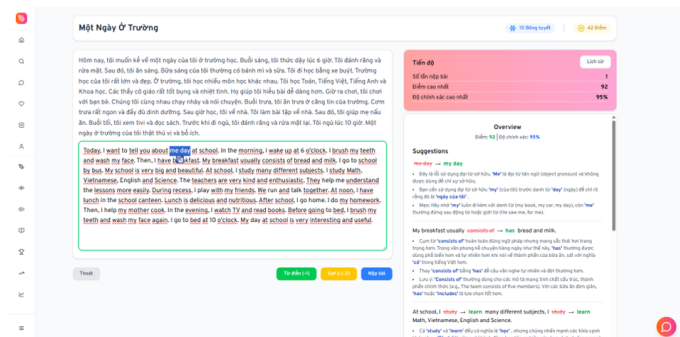


Fig. 7: Speech and writing interface, AI scoring suggestions.

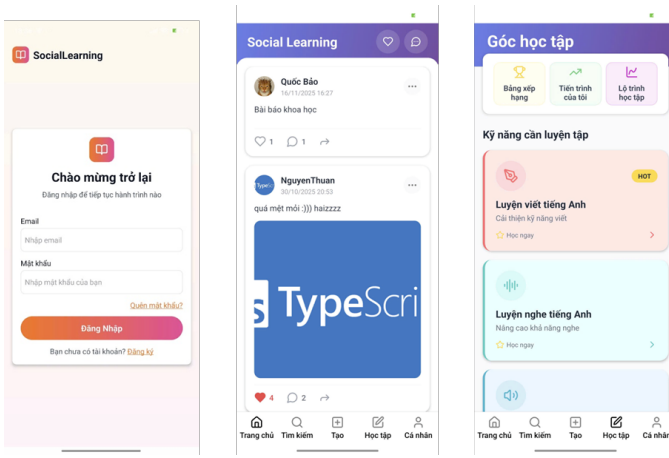


Fig. 8: Mobile user interface.

C. Preliminary results on speech-to-text performance

To evaluate the technical feasibility and accuracy of the pronunciation assessment module (integrated via Cloudinary and Gemini API), we conducted a quantitative experiment. The dataset consists of 30 distinct audio samples recorded by users, categorized into three difficulty levels: Basic (short sentences), Intermediate (complex structures), and Advanced (academic vocabulary).

1) *Metrics and Results:* We measured *Accuracy* (based on Word Error Rate) and *Latency*. Fig. 9 visualizes the performance distribution across the three levels.

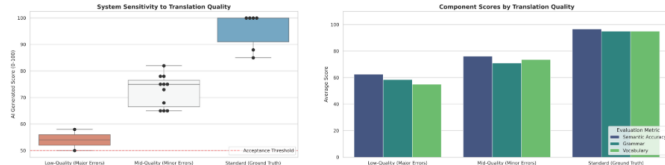


Fig. 9: Evaluation of Speech-to-Text accuracy. (a) Average accuracy declines as complexity increases (100% for Basic vs. 60.2% for Advanced). (b) Distribution of scores shows higher variance and outliers in Advanced levels.

As illustrated in Fig. 9(a), the experimental data reveals a strong correlation between sentence complexity and system performance:

- **Basic Level:** The system achieved absolute precision (100% accuracy) with an average latency of 989 ms, confirming its effectiveness for beginners.
- **Intermediate Level:** Performance remains reliable with an average accuracy of 72.5% and latency of 1251 ms.
- **Advanced Level:** Accuracy drops to 60.2% with a latency of 1592 ms. The wider distribution shown in Fig. 9(b) indicates challenges with acoustic similarities in specialized terminology.

D. Experimental Evaluation of AI Writing Assistant

To assess the effectiveness of the Gemini-powered writing feedback, we analyzed 30 essays across three proficiency levels. The system evaluates essays based on three core criteria: *Grammar*, *Vocabulary Richness*, and *Coherence*.

1) *Evaluation Results:* Fig. 10 presents the component scores assigned by the AI agent.

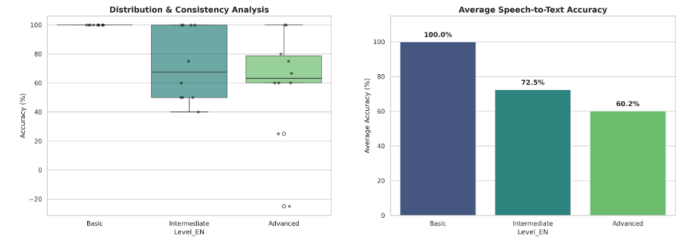


Fig. 10: AI Writing Feedback scores by component. The system effectively differentiates vocabulary and coherence proficiency between Basic and Advanced learners.

The analysis indicates:

- **Grammar Accuracy:** The system identifies grammatical errors with high precision. Basic essays received higher average scores ($\approx 90\%$) due to simpler sentence structures compared to Advanced essays ($\approx 82\%$).
- **Vocabulary Differentiation:** The AI effectively distinguishes proficiency levels. As shown in Fig. 10, Advanced users scored significantly higher in Vocabulary ($\approx 85\%$) compared to Basic users ($\approx 60\%$), validating the AI's ability to recognize lexical sophistication.

E. Evaluation of Mastery Scoring Algorithm

To validate the pedagogical efficacy and technical stability of the Mastery Score algorithm defined in Section II, a stochastic simulation was conducted over $N = 50$ iterations. The simulation aimed to measure the convergence speed and discrimination capability of the scoring logic under the parameter set: $P_{correct} = 5$ and $P_{wrong} = 3$.

Three distinct user profiles were simulated to represent different proficiency levels:

- **User A (High Proficiency):** 90% accuracy probability.
- **User B (Average Proficiency):** 60% accuracy probability.
- **User C (Low Proficiency):** 30% accuracy probability.

1) *Quantitative Analysis of Convergence:* The simulation results, as visualized in Fig. 11, demonstrate that the algorithm successfully differentiates learning behaviors based on input accuracy:

- **Convergence Speed (Efficiency):** User A reached the mastery threshold (100 points) in 22 attempts. In contrast, User B required 35 attempts to achieve the same milestone. This indicates that the system implicitly enforces a repetition ratio of $\approx 1.6x$ for average learners compared to high performers, ensuring adequate reinforcement for weaker concepts.
- **Penalty Effectiveness (Discrimination):** User C, with a low accuracy rate (30%), failed to reach the mastery threshold within the simulation window (score remained < 10). This quantitatively confirms that the penalty weight ($P_{wrong} = 3$) effectively prevents progression through random guessing or insufficient knowledge retention.

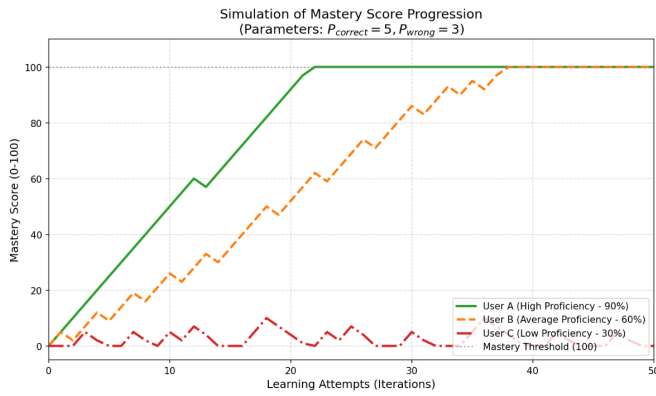


Fig. 11: Simulation of Mastery Score progression. The graph illustrates distinct learning trajectories: User A achieves linear growth, User B exhibits fluctuations due to penalty enforcement, and User C is bounded at lower scores, validating the algorithm's discriminative ability.

2) *Qualitative Analysis of Scoring Logic*: Beyond numerical metrics, the trajectory patterns in Fig. 11 reveal critical behavioral characteristics designed for learner retention:

- **Boundary Resilience**: As observed in the trajectory of User C, the score frequently hits the lower bound. The implementation of the $\max(0, M_{old} - P_{wrong})$ function ensures that scores never drop into negative values. Qualitatively, this design decision protects beginner learners from demotivation during the early acquisition phase.
- **Consistency Enforcement**: The "sawtooth" pattern observed in User B's trajectory confirms that the penalty mechanism acts as a "soft barrier." A user cannot achieve mastery solely by accumulating points over time; they must demonstrate a consistent streak of correct answers to overcome the penalty setbacks.

In conclusion, the experimental data verifies that the proposed Mastery Score mechanism functions as an adaptive filter, effectively distinguishing between different levels of learner proficiency while maintaining a pedagogical balance between reward and penalty.

VI. CONCLUSION AND FUTURE WORK

A. Conclusion

Social-Learning is a social learning system for learning the English language (Web and Mobile). With the use of a separated Client-Server architecture, the system can incorporate the latest technologies, including Next.js, React Native, Supabase, and MongoDB... The greatest contribution of the research is the development and execution of an Adaptive Learning Loop. The system also uses the Google Gemini API to counter the absence of personalization in traditional applications; thus, it does not deliver static learning material. This AI loop, combined with a well-developed social functionality (adding friends, posting) and multimedia communication (messaging, video calls), allows forming an active, strongly interactive classroom. The results of the implementation are evidence of the feasibility and efficiency of the offered model with regard

to the necessity of the English learning platform, which is not only community-focused but also highly personalized.

B. Future Work

According to the obtained outcomes, the research group suggests several ways to develop the system further:

Optimizing AI Performance: Enhance the speed of AI capabilities. Optimize the messages to the Gemini API to make the writing evaluation more accurate (Figure 5) at a lower API cost. **Improving Analytics**: Adding to the "Your Progress" page, include more of the detailed analytical charts about the learning activities of the users, to enable them to easily learn what they were strong in and weak in, and how much time they spent on the particular skills. **System Performance and Security**: Optimize system components such as Nginx and API performance to process the high traffic, especially during peak learning periods. Enhance security controls, particularly user data protection, to address high security requirements and system safety.

VII. REFERENCES

- [1]. Statista Research Department, "Number of language learning app users worldwide from 2016 to 2024," Statista, 2024. [Online]. Available: <https://www.statista.com>
- [2]. L. S. Vygotsky, *Mind in Society: The Development of Higher Psychological Processes*, Harvard University Press, 1978.
- [3]. TypeScript for JavaScript Programmers. Available : <https://www.typescriptlang.org/docs/handbook/typescript-in-5-minutes.html>
- [4]. Riva, M. (2022). *Real-World Next.js: Build scalable, high-performance, and modern web applications using Next.js, the React framework for production*. Packt Publishing Ltd
- [5]. O'Reilly Media, Inc. (2020) Chapter 1: What Is React Native? [Online]. Available: <https://www.oreilly.com/library/view/learning-react-native/9781491929049/ch01.html>
- [6]. Gemini API reference. Available: <https://ai.google.dev/api?hl=vi>
- [7]. Stones, R., & Matthew, N. (2006). *Beginning databases with PostgreSQL: From novice to professional*. Apress.
- [8]. How to Use Supabase: Complete Beginner's Guide to Build Apps with Database, Auth & Storage <https://supabase.com/docs>
- [9]. Client-server overview. Available: https://developer.mozilla.org/en-US/docs/Learn_web_development/Extensions/Server-side/First_steps/Client-Server_overview
- [10]. Accuracy evaluation metric. Available: <https://www.ibm.com/docs/en/ws-and-kc?topic=metrics-accuracy>
- [11]. Mastery scores, completed, passed and failed states. Available: <https://support.civica-learning.com/hc/en-us/articles/360000513526-Explained-Mastery-scores-completed-passed-and-failed-states>